

05.05.2026



**FATİH
SULTAN
MEHMET**
VAKIF ÜNİVERSİTESİ

T.C.

FATİH SULTAN MEHMET VAKIF UNIVERSITY

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

PROGRAMMING FOR DATA ENGINEERING

PROJECT REPORT

PREPARED BY

**Elif Nisa BAŞ 2421021019
Nisa AKTAŞ. 2321021023
Zehra ŞAHİNER 2421021022
Zehranur ÖZDOĞAN 2421021738**

COURSE INSTRUCTOR

**Asst. Prof. Dr. Abdulkерim Mohammed YİBRE
Res. Asst. Zeliha Kaya AKÇELİK**

2025-2026 SPRING TERM

1. Project Overview

The Course Follow-Up System is a desktop application developed using Python and the Tkinter GUI framework, backed by a SQLite relational database. The system provides a multi-role academic management platform supporting three distinct user types: Administrator, Teacher, and Student. Each role is granted access to a specific set of features tailored to their responsibilities within an academic institution.

The application follows a layered architecture pattern consisting of a data access layer (DatabaseManager), a business logic layer (Service classes), model classes, and a presentation layer (UI windows). This separation of concerns ensures maintainability and extensibility of the codebase.

2. System Architecture

2.1 Project Structure

The project is organized into the following directory structure:

Directory / File	Description
main.py	Application entry point; initializes services and login window
database/db_manager.py	DatabaseManager class; creates tables and manages connections
models/	Data model classes: Student, Instructor, Course, Enrollment, Grade, Attendance, User, Message, Notification, Schedule
services/	Business logic layer: AuthManager, StudentService, InstructorService, CourseService, EnrollmentService, GradeService, AttendanceService, ReportService, MessageService, NotificationService, ScheduleService
ui/	GUI windows: LoginWindow, AdminWindow, TeacherWindow, StudentWindow, SharedComponents

2.2 Technology Stack

Technology	Purpose
Python 3.x	Primary programming language
Tkinter + ttk	Desktop GUI framework
SQLite3	Embedded relational database
ReportLab	PDF transcript generation

3. Class Inventory

The system contains 20+ classes, with a minimum of 16 non-UI classes as required. These are divided into three categories:

3.1 Model Classes (10 classes)

Model classes represent the core data entities of the system:

- Student — student_id, name, surname, department; get_full_name()
- Instructor — instructor_id, name, surname, branch; get_full_name()
- Course — course_id, course_name, course_code, credit
- Enrollment — enrollment_id, student_id, course_id, grade, status
- Grade — student_id, course_id, grade, status; get_letter_grade() → AA/BA/BB/CB/CC/FF
- Attendance — attendance_id, student_id, course_id, total_hours, absent_hours; calculate_percentage()
- User — user_id, username, password, role, student_id, instructor_id
- Message — message_id, sender_id, receiver_id, subject, body, is_read, sent_at
- Notification — notification_id, user_id, title, body, is_read, created_at
- Schedule — schedule_id, course_id, day_of_week, start_time, end_time, classroom

3.2 Service Classes (11 classes)

Service classes implement the business logic and all database operations:

- DatabaseManager — creates all tables, manages SQLite connection, inserts default admin
- AuthManager — login() with User model integration, create_user(), delete_user(), get_all_users()
- StudentService — CRUD operations for students; get_student_by_id() returns Student object
- InstructorService — CRUD operations for instructors
- CourseService — CRUD operations for courses; assign_instructor(), get_courses_by_instructor()
- EnrollmentService — enroll_student(), update_grade(), get_enrollments_by_student/course()
- GradeService — get_grades_by_student() returns Grade objects with letter grades; calculate_gpa() on 4.0 scale
- AttendanceService — add/update/delete attendance; attendance percentage calculation via SQL CASE
- ReportService — get_summary() for dashboard statistics; student and course reports
- MessageService — send_message(), get_inbox(), get_sent(), mark_as_read()
- NotificationService — send_notification(), send_bulk_notification(), mark_all_as_read()
- ScheduleService — add/update/delete schedules; get by student, instructor, or course

3.3 UI Classes (5 classes)

UI classes handle the graphical interface. These are excluded from the minimum class count requirement:

- LoginWindow — credential entry form with Enter key binding and role-based redirection
- AdminWindow — full CRUD management for students, teachers, courses, enrollments
- TeacherWindow — schedule view, grade management, attendance tracking, messaging
- StudentWindow — dashboard, transcript, attendance status, messaging, notifications
- SharedComponents (StudentProfileView, StudentDetailView) — reusable profile and PDF export views

4. Authentication and Dynamic Role Management

The entry point of the system is the Login Module (LoginWindow), which handles security and user redirection.

4.1 Database-Driven Authentication

When a user enters their credentials, the LoginWindow queries the users table via AuthManager.login(). The AuthManager constructs a User model object from the database row, then returns a dictionary containing the user's role, unique academic ID (student_id or instructor_id), username, and user_id.

4.2 Dynamic Redirection

Upon successful login, the on_login_success callback in main.py clears the existing UI widgets and dynamically instantiates one of three panels based on the role field:

- role = 'admin' → AdminWindow
- role = 'teacher' → TeacherWindow
- role = 'student' → StudentWindow

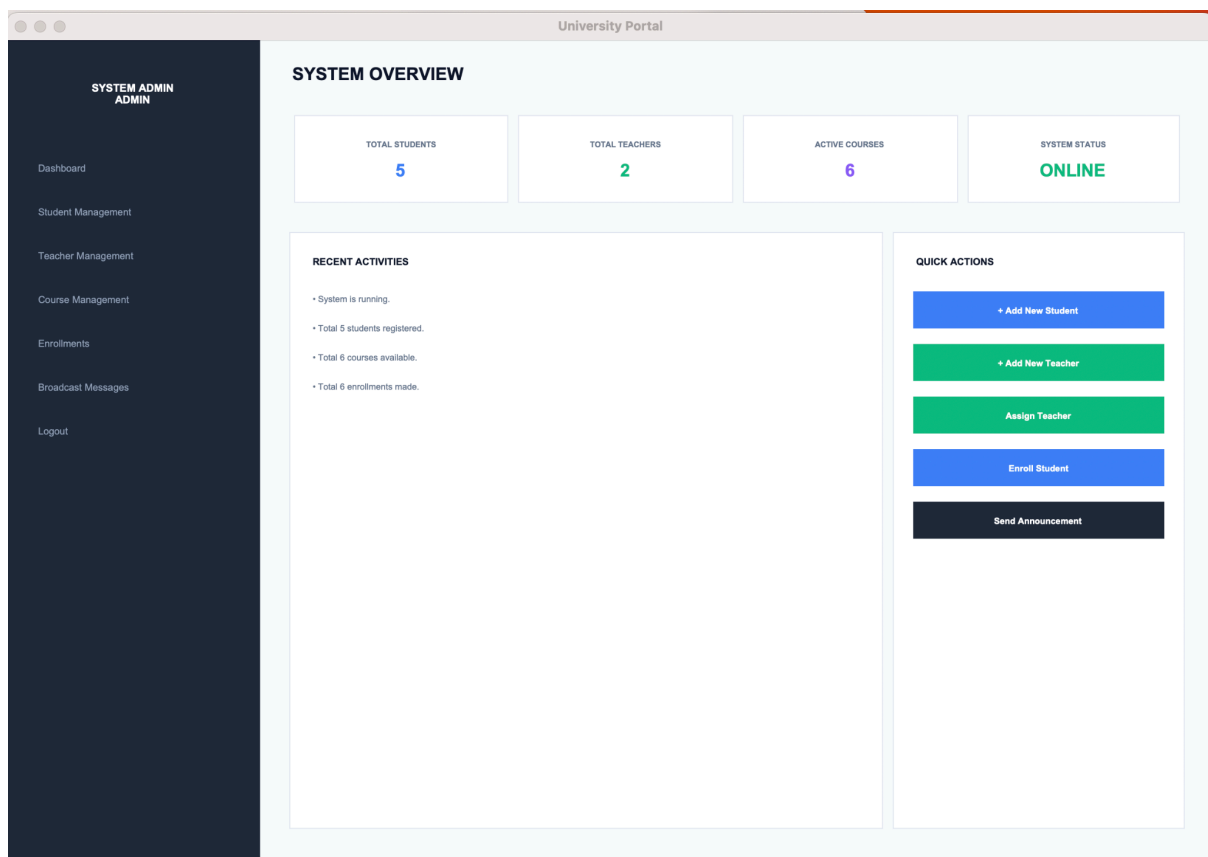
This ensures users can only access features permitted by their authorization level. The callback reference is stored on the root window (root.on_login_success_callback) to support the logout-and-return flow from any panel.

5. Primary Management Panels

5.1 Administrative Panel (AdminWindow)

The AdminWindow provides full system control through the following modules:

- Student Management — Register new students (name, TC number, department); system generates a unique student ID (2421000–2421999 range); username and TC-based password are auto-created in the users table; supports delete and view-more profile
- Teacher Management — Add instructors with ID, name, branch; auto-creates teacher login credentials; assign instructors to courses
- Course Management — Add courses (name, code, credit) with auto-generated course IDs; delete courses with cascading enrollment removal
- Enrollment Management — Enroll students into courses via dropdown selection; view all enrollments
- Student Profile & PDF Export — View detailed student transcript with GPA and course list; export to PDF via ReportLab through the shared StudentDetailView component
- System Dashboard — Real-time statistics (total students, teachers, courses, enrollments) via ReportService.get_summary()



5.2 Instructor Panel (TeacherWindow)

The TeacherWindow provides tools for course delivery and student management:

- **Schedule** — Displays the instructor's assigned courses with day/time; filterable by course and day
- **Students** — Lists all enrolled students across the instructor's courses; searchable and filterable; links to full student profile with PDF export
- **Grade Management** — Select a course, view enrolled students, enter grades (0–100); grade is saved and letter grade is computed using the Grade model
- **Messages** — Inbox populated from the messages table; reply functionality; view academic data of message sender
- **Announcements** — Broadcast center with per-course and per-student checkbox selection; message saved to students_db for in-session viewing

The screenshot shows a web application titled "University Portal" running in a browser window. The interface has a dark blue sidebar on the left with the text "WELCOME, YETER.ARTIK" and a list of navigation links: "My Courses", "Dashboard & Grades", "Attendance", "Transcript", "Messages", "Notifications", "My Profile", and "Logout". The main content area is titled "MY COURSES" and displays a table of enrolled courses. The table has five columns: "Course Code", "Course Name", "Credit", "Grade", and "Status". The data rows are as follows:

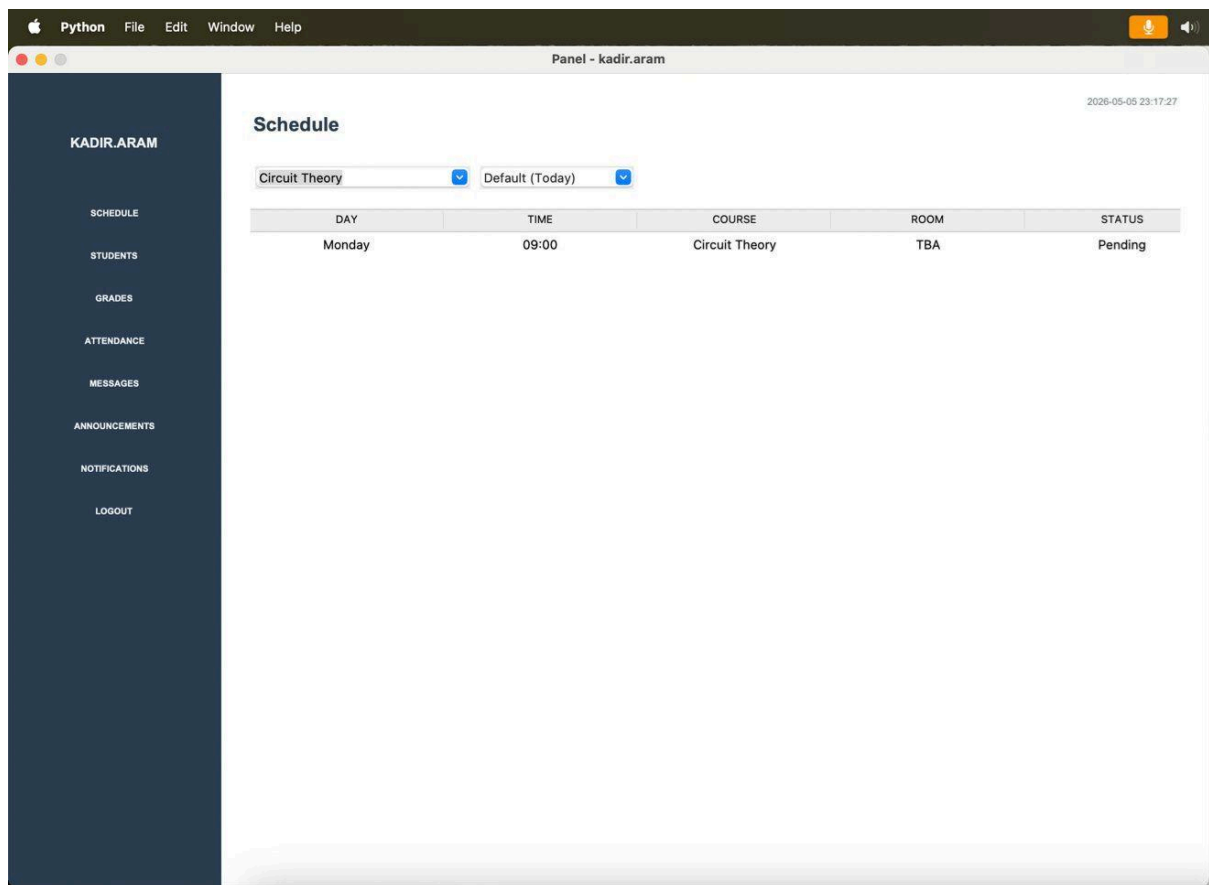
Course Code	Course Name	Credit	Grade	Status
CENG201-A	Algorithms-A	5	N/A	enrollec
CENG201-B	Algorithms-B	5	N/A	enrollec
CENG301-A	Database-A	4	N/A	enrollec
CENG301-B	Database-B	4	N/A	enrollec
LAW202	Criminal Law	4	N/A	enrollec
BUS105	Marketing	3	N/A	enrollec

The date "Tuesday, 05 May 2026" is displayed in the top right corner of the main content area.

5.3 Student Panel (StudentWindow)

The StudentWindow gives students read access to their academic information:

- My Courses — Displays enrolled courses with grade and status from the enrollments table
- Academic Dashboard — Shows cumulative GPA (4.0 scale) and enrolled course count via GradeService
- Transcript — Full course list with grades; Export PDF button triggers StudentDetailView.export_pdf() from SharedComponents
- Messages — Send messages to course instructors via MessageService
- Notifications — View and mark-as-read notifications from the notifications table
- My Profile — Displays student name, ID, department from the Student model object



6. Secure Data Management

6.1 Database Schema

All system data is stored in a centralized SQLite database (course_follow_up.db), which is automatically created on first run by DatabaseManager. The schema consists of 8 tables:

Table	Key Columns	Foreign Keys
students	student_id (PK), name, surname, department	—
instructors	instructor_id (PK), name, surname, branch	—
courses	course_id (PK), course_name, course_code, credit, instructor_id	→ instructors
enrollments	student_id + course_id (PK), grade, status	→ students, courses
attendance	attendance_id (PK), student_id, course_id, total_hours, absent_hours	→ students, courses
users	user_id (PK), username, password, role, student_id, instructor_id	→ students, instructors
messages	message_id (PK), sender_id, receiver_id, subject, body, is_read	→ users
notifications	notification_id (PK), user_id, title, body, is_read	→ users
schedules	schedule_id (PK), course_id, day_of_week, start_time, end_time	→ courses

6.2 Data Integrity

All foreign key relationships are enforced at the SQLite level using PRAGMA foreign_keys = ON, which is activated on every connection. Cascading DELETE rules are applied throughout:

- Deleting a student removes all their enrollments, attendance records, and user account
- Deleting a course removes all related enrollments, attendance, and schedule entries
- Deleting an instructor sets the instructor_id in courses to NULL (SET NULL), preserving course data

6.3 GPA Calculation

The `GradeService.calculate_gpa()` method computes a weighted GPA on the 4.0 scale, using course credits as weights:

Score Range	GPA Points
90 – 100	4.0
85 – 89	3.5
80 – 84	3.0
75 – 79	2.5
70 – 74	2.0
65 – 69	1.5
60 – 64	1.0
0 – 59	0.0

7. Shared Components and Code Reuse

The `SharedComponents` module (`ui/SharedComponents.py`) contains two reusable view classes that eliminate code duplication across `AdminWindow`, `TeacherWindow`, and `StudentWindow`:

7.1 StudentProfileView

A static view used in earlier iterations for displaying basic student info, grades table, and attendance table from a pre-loaded data dictionary.

7.2 StudentDetailView

A fully database-integrated static class that accepts a `student_id`, a `services` dictionary, a `colors` dictionary, and a `back_callback` function. It:

- Fetches the `Student` object via `StudentService`
- Calculates GPA via `GradeService`
- Loads enrollment data via `EnrollmentService`
- Renders a complete transcript table with stats cards
- Provides an `EXPORT PDF` button that calls `export_pdf()` using `ReportLab` to generate a formatted A4 PDF file

This component is called identically from `AdminWindow.show_student_profile()`, `TeacherWindow.show_student_full_profile()`, and `StudentWindow.show_transcript()`, ensuring consistent behavior and a single point of maintenance.

8. PDF Transcript Export

The PDF export functionality is implemented in `StudentDetailView.export_pdf()` using the ReportLab library. The generated document includes:

- Document title and institution name centered at the top
- Student information block: full name, student ID, department, cumulative GPA
- Course transcript table: course code, course name, credit hours, numeric grade, status
- Automatic page overflow handling — new pages are created if the transcript exceeds one page

The user is presented with a native file save dialog (`tkinter.filedialog.asksaveasfilename`) to choose the save location. The default filename is `transcript_{name}_{surname}.pdf`.

9. Conclusion

The Course Follow-Up System successfully implements a multi-role academic management platform with a clean layered architecture. The system satisfies all project requirements:

- Minimum 16 non-UI classes implemented (10 models + 12 services including `DatabaseManager`)
- Full GUI using Tkinter with role-based access control
- SQLite database with foreign key constraints and cascading deletes
- PDF export via ReportLab
- Shared component architecture for code reuse
- Git-based version control with collaborative development

The modular design allows straightforward extension — new features such as a full schedule management UI, real-time messaging, or advanced reporting can be added by implementing new service methods and connecting them to existing or new UI panels without modifying the database schema or core logic.